

Java Narrowing Conversion (Explicit Type Casting) & Data Loss Risk

Learning Objectives

After this topic, you will be able to:

- Understand what narrowing conversion is.
 - Perform explicit type casting.
 - Know when narrowing is required.
 - Understand the risk of data loss.
 - Avoid common mistakes in interviews.
-

1. What is Narrowing Conversion?

Narrowing conversion is the process of converting a **larger data type** into a **smaller data type**.

Unlike widening conversion, **Java does not perform narrowing automatically** because it may cause **data loss**.

Example:

```
double price = 99.99;  
int amount = (int) price;
```

Here, (int) is called **explicit casting**.

2. Why is Casting Required?

A smaller data type cannot always hold all the values of a larger data type.

Example:

```
long number = 5000000000L;  
int value = (int) number;
```

The int cannot store such a large value, so data is lost.

3. Common Narrowing Conversions

```
double → float  
float → long  
long → int  
int → short  
short → byte  
int → char
```

All require **explicit casting**.

4. Examples

Example 1: double → int

```
double salary = 45000.75;  
int s = (int) salary;  
  
System.out.println(s);
```

Output:

```
45000
```

The decimal part (.75) is discarded.

Example 2: int → byte

```
int num = 100;  
byte b = (byte) num;  
  
System.out.println(b);
```

Output:

```
100
```

No data is lost because 100 fits within the byte range.

Example 3: Data Loss

```
int num = 130;  
byte b = (byte) num;  
  
System.out.println(b);
```

Output:

```
-126
```

Reason: 130 is outside the byte range (-128 to 127), causing overflow.

5. Data Loss Risk

Narrowing conversion can cause:

- Loss of decimal values
- Overflow or underflow
- Unexpected results

Always ensure the value fits into the target type before casting.

6. Common Mistakes

✗ Forgetting to cast:

```
double d = 5.5;  
int x = d;
```

Compiler Error

✓ Correct:

```
int x = (int) d;
```

✗ Assuming casting rounds the value.

```
double d = 9.99;  
int x = (int) d;
```

Output:

```
9
```

Casting **truncates** the decimal part; it does **not** round.

7. Best Practices

- Cast only when necessary.
 - Check that the value is within the target type's range.
 - Be cautious when converting floating-point values to integers.
-

8. Tricky Interview Questions

Q1. What is narrowing conversion?

Answer: Converting a larger data type to a smaller one using explicit casting.

Q2. Is narrowing automatic?

Answer: No. It requires an explicit cast.

Q3. Why is casting required?

Answer: Because narrowing may result in data loss.

Q4. Does casting round decimal values?

Answer: No. It simply removes (truncates) the fractional part.

Q5. What happens if an int value is too large for a byte?

Answer: The value overflows, producing an unexpected result.

9. Output-Based Questions

Question 1

```
double d = 10.8;
int x = (int) d;

System.out.println(x);
```

Output:

```
10
```

Question 2

```
int num = 130;  
byte b = (byte) num;  
  
System.out.println(b);
```

Output:

-126

Question 3

```
float f = 25.9F;  
int i = (int) f;  
  
System.out.println(i);
```

Output:

25

10. Cheat Sheet

Conversion	Automatic?	Cast Required?	Data Loss Possible?
double → int	✗ No	✓ Yes	✓ Yes
long → int	✗ No	✓ Yes	✓ Yes
int → byte	✗ No	✓ Yes	✓ Yes

float → int	 No	 Yes	 Yes
-------------	--	---	---

Key Points to Remember

- **Widening** = Smaller → Larger (**Automatic**)
- **Narrowing** = Larger → Smaller (**Explicit Cast Required**)
- Narrowing can cause **data loss** or **overflow**.
- Casting **does not round** values; it **truncates** the decimal part.
- Always verify that the value fits in the destination type before casting.